Análisis Asintótico - Cotas

Cota superior O

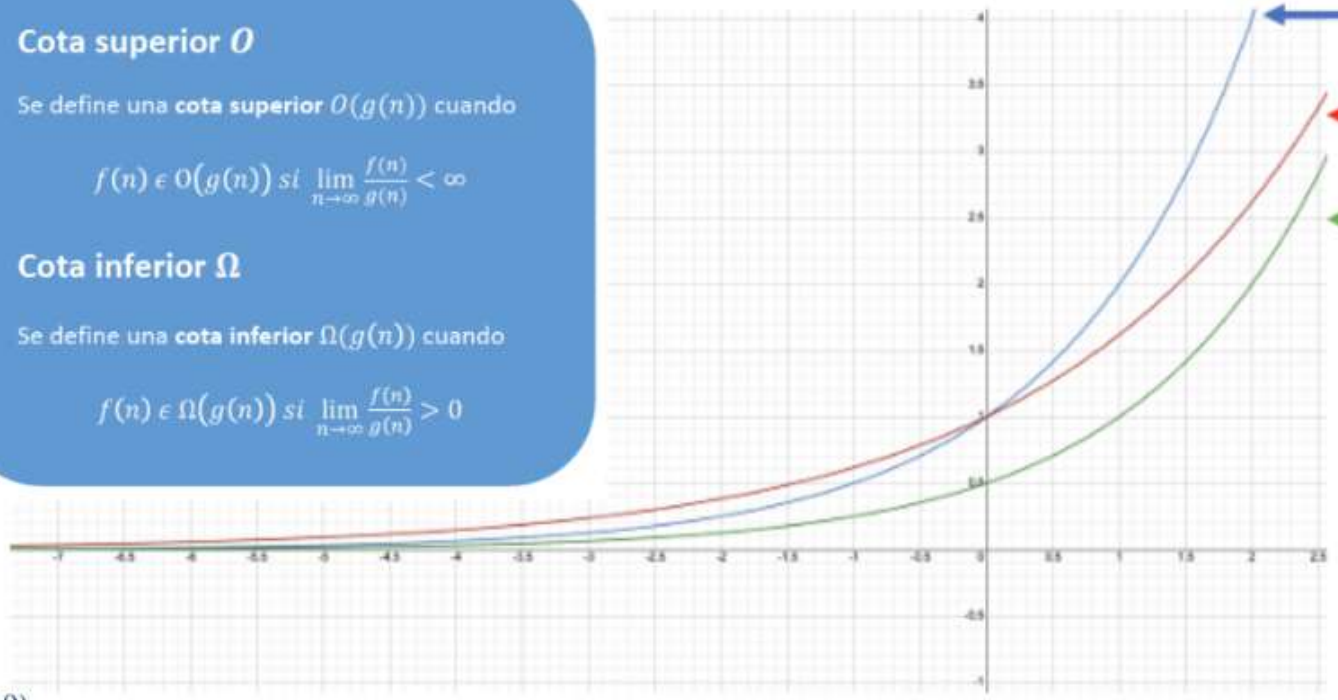
Se define una **cota superior** $O(g(n))$ cuando

$$f(n) \in O(g(n)) \; si \; \lim_{n\to\infty} \frac{f(n)}{g(n)} < \infty$$

Cota inferior Ω

Se define una **cota inferior** $\Omega(g(n))$ cuando

$$f(n) \in \Omega(g(n)) \; si \; \lim_{n\to\infty} \frac{f(n)}{g(n)} > 0$$



$O(g(n))$ *Cota superior*

$\Theta(g(n))$ *Orden exacto*

$\Omega(g(n))$ *Cota inferior*

Complejidad de Fibonacci:

$$O(g(n)) = 2^n$$

$$\theta(g(n)) = \left(\frac{1+\sqrt{5}}{2}\right)^n$$

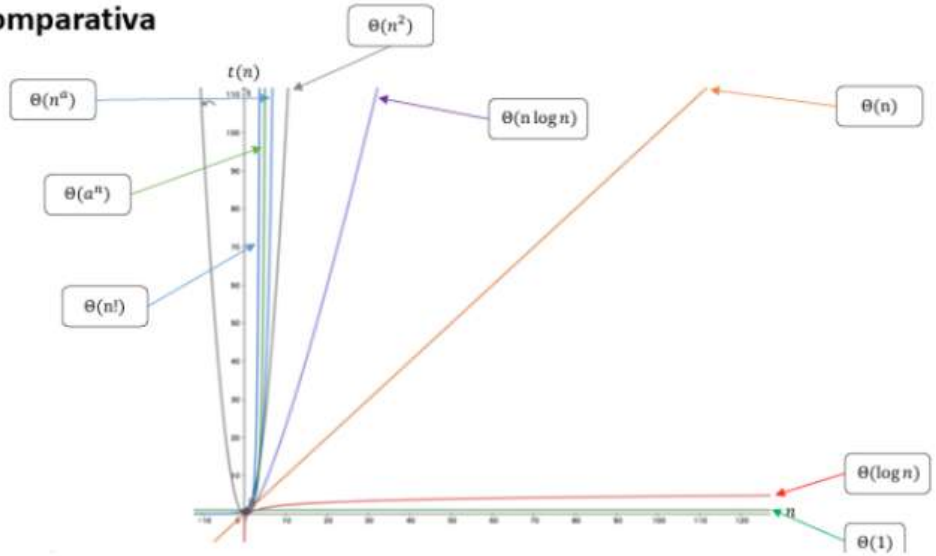
$$\Omega(g(n)) = 2^{n/2}$$

Adaptado de Romero (2019)

Análisis Asintótico - Notación

- Jerarquía

comparativa

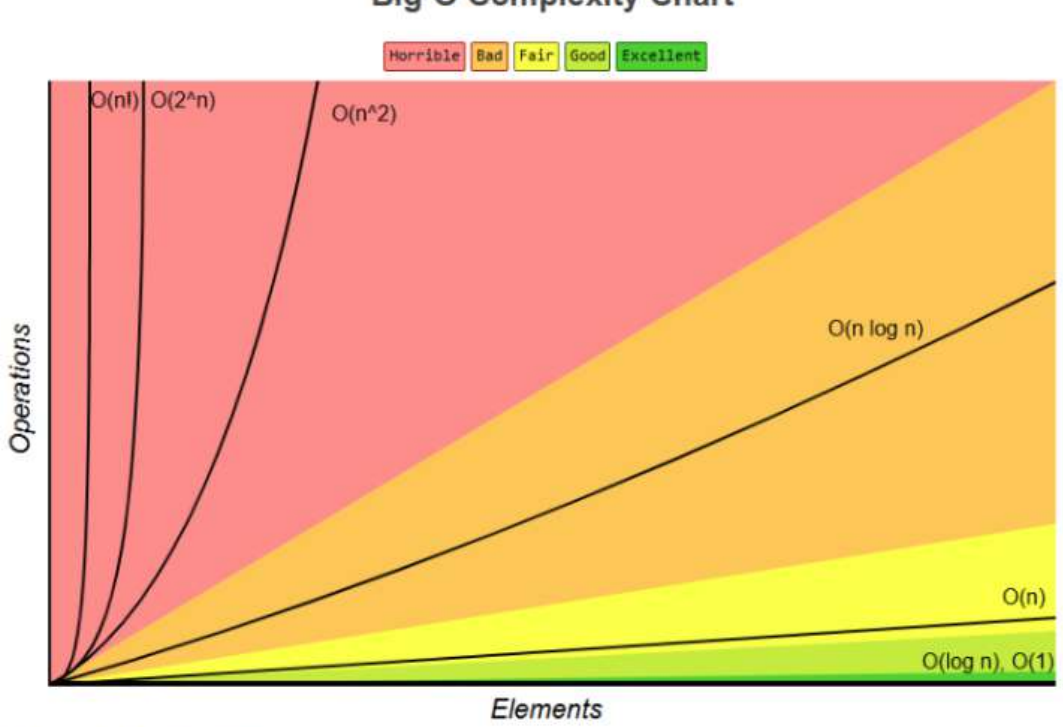


$\Theta(1)$	Orden constante
$\Theta(\log n)$	Orden logarítmico
$\Theta(\mathrm{n})$	Orden lineal
$\Theta(\mathrm{n} \log n)$	Orden cuasi-lineal
$\Theta(n^2)$	Orden cuadrático
$\Theta(n^3)$	Orden cúbico
$\Theta(n^a)$	Orden polinómico ($a > 3$)
$\Theta(a^n)$	Orden exponencial ($a > 1$)
$\Theta(\mathrm{n}!)$	Orden factorial

$$\boldsymbol{\Theta(1) < \Theta(log\, n) < \Theta(n) < \Theta(\mathrm{n}\, log\, n) < \Theta(n^2) < \Theta(n^3) < \Theta(n^a) < \Theta(a^n) < \Theta(\mathrm{n}!) < \Theta(n^n)}$$

Adaptado de Romero (2019)

Análisis Asintótico - Notación

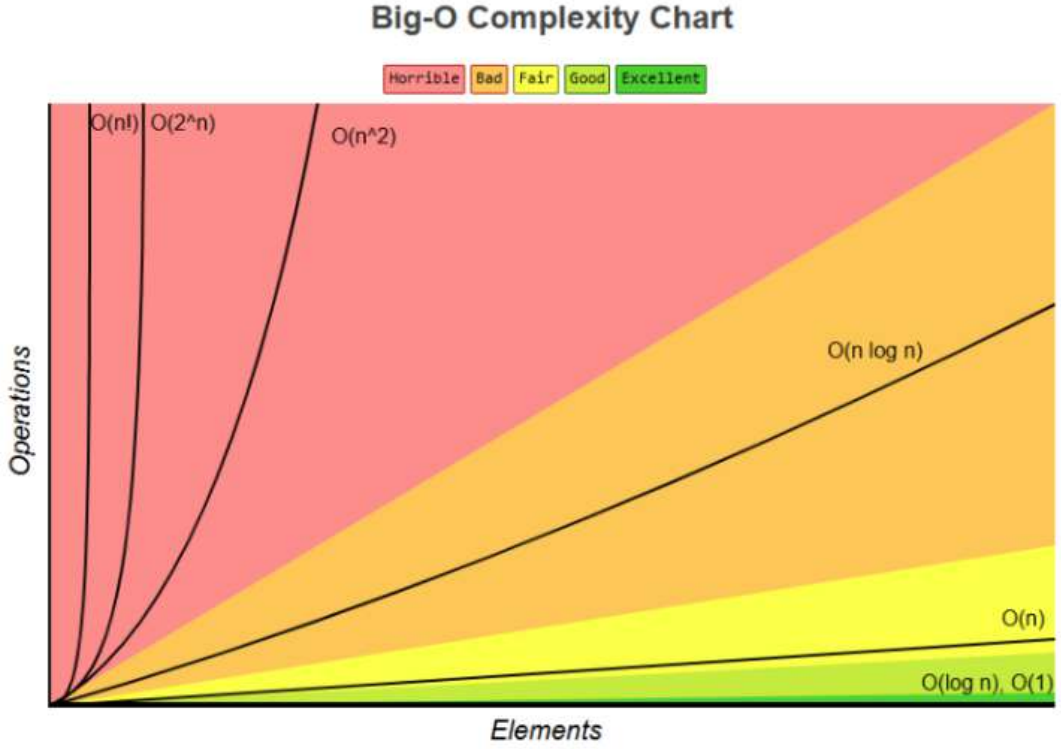


Common Data Structure Operations

Data Structure	Time Complexity								Space Complexity
	Average				Worst				Worst
	Access	Search	Insertion	Deletion	Access	Search	Insertion	Deletion	
Array	Θ(1)	Θ(n)	Θ(n)	Θ(n)	O(1)	O(n)	O(n)	O(n)	O(n)
Stack	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Queue	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Singly-Linked List	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Doubly-Linked List	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Skip List	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n log(n))
Hash Table	N/A	Θ(1)	Θ(1)	Θ(1)	N/A	O(n)	O(n)	O(n)	O(n)
Binary Search Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n)
Cartesian Tree	N/A	Θ(log(n))	Θ(log(n))	Θ(log(n))	N/A	O(n)	O(n)	O(n)	O(n)
B-Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
Red-Black Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
Splay Tree	N/A	Θ(log(n))	Θ(log(n))	Θ(log(n))	N/A	O(log(n))	O(log(n))	O(log(n))	O(n)
AVL Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
KD Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n)

Adaptado de BigOCheat

Análisis Asintótico - Notación



Adaptado de BigOCheat

Array Sorting Algorithms

Algorithm	Time Complexity			Space Complexity
	Best	Average	Worst	Worst
Quicksort	Ω(n log(n))	θ(n log(n))	O(n^2)	O(log(n))
Mergesort	Ω(n log(n))	θ(n log(n))	O(n log(n))	O(n)
Timsort	Ω(n)	θ(n log(n))	O(n log(n))	O(n)
Heapsort	Ω(n log(n))	θ(n log(n))	O(n log(n))	O(1)
Bubble Sort	Ω(n)	θ(n^2)	O(n^2)	O(1)
Insertion Sort	Ω(n)	θ(n^2)	O(n^2)	O(1)
Selection Sort	Ω(n^2)	θ(n^2)	O(n^2)	O(1)
Tree Sort	Ω(n log(n))	θ(n log(n))	O(n^2)	O(n)
Shell Sort	Ω(n log(n))	θ(n(log(n))^2)	O(n(log(n))^2)	O(1)
Bucket Sort	Ω(n+k)	θ(n+k)	O(n^2)	O(n)
Radix Sort	Ω(nk)	θ(nk)	O(nk)	O(n+k)
Counting Sort	Ω(n+k)	θ(n+k)	O(n+k)	O(k)
Cubesort	Ω(n)	θ(n log(n))	O(n log(n))	O(n)

Análisis Asintótico - Propiedades

- Constantes

Teorema 1.3.1: Regla de constantes

Sea $f : \mathbb{N} \to \mathbb{R}_{\geq 0}$. Si $c \in \mathbb{R}_{>0}$, entonces $O(f) = O(cf)$.

Análisis Asintótico - Propiedades

- Sumas

Teorema 1.3.2: Subsunción por suma

Sea $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$. Si $g \in O(f)$, entonces $O(f + g) = O(f)$, en donde $f + g$ denota la suma punto a punto de f y g (i.e., $(f + g)(n) = f(n) + g(n)$).

Análisis Asintótico - Propiedades

- Relación de orden

Teorema 1.3.4

Sean $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$:

1. $O(f) = O(g)$ sii $f \in O(g)$ y $g \in O(f)$.
2. $O(f) \subset O(g)$ sii $f \in O(g)$ y $g \notin O(f)$.

Análisis Asintótico - Propiedades

- Suma y Producto

Teorema 1.3.5: Reglas de suma y producto

Sean $f, f_1, g, g_1 : \mathbb{N} \to \mathbb{R}_{\geq 0}$:

1. $O(f+g) = O(f \uparrow g)$, en donde la suma y el máximo se interpretan punto a punto.
2. Si $f_1 \in O(f)$ y $g_1 \in O(g)$, entonces $f_1 g_1 \in O(fg)$, en donde el producto se interpreta punto a punto.

Tipo de operación	Regla asintótica	Ejemplo
$f(n) + g(n)$	$O(\max(f, g))$	$O(n + \log n) = O(n)$
$f(n) \cdot g(n)$	$O(f \cdot g)$	$O(n) \cdot O(\log n) = O(n \log n)$

Rocha (2025)

¿Cuál es la complejidad temporal?

- Depende...

Análisis Asintótico - Ejemplo

```python
def list_sum(A):
    sum = 0
    for i in range(len(A)):
        sum = sum + A[i]
    return sum
```

1. `sum = 0`

- Operación de asignación.
- 🕒 **Tiempo constante = 1 unidad**

2. `range(len(A))`

- `len(A)` se ejecuta una vez → constante.
- `range(n)` genera una secuencia de n elementos → $n + 1$ operaciones (considerando desde 0 hasta $n - 1$).
- 🕒 **Costo total = $2(n + 1)$**

3. Bucle `for i in range(...)`

- Se ejecuta **n veces** (una por cada índice de A).

Dentro del bucle:

`sum = sum + A[i]`

- **Acceso a A[i]** → 1 operación
- **Suma** → 1 operación
- **Asignación** → 1 operación
- 🕒 **Total por iteración = 3 operaciones**
- **Total del bucle = $3n$**

4. `return sum`

- 🕒 **Una sola operación → 1**

Sumando todo:

Componente	Operaciones
Inicialización `sum = 0`	1
Construcción de `range`	$2(n+1)$
Suma en el bucle	$3n$
Return final	1
Total (Tsum)	$1 + 2(n+1) + 3n + 1 = 5n + 4$

Simplificando con notación asintótica:

$$T(n) = 5n + 4 = C_1 \cdot n + C_2 \Rightarrow \boxed{O(n)}$$

¿Cuál es la complejidad temporal?

- Depende...

Análisis Asintótico - Ejemplo

```python
def list_sum(A):
    sum = 0
    for i in range(len(A)):
        sum = sum + A[i]
    return sum
```

1. `sum = 0`

- Operación de asignación.
- 🕒 **Tiempo constante = 1 unidad**

2. `range(len(A))`

- `len(A)` se ejecuta una vez → constante.
- `range(n)` genera una secuencia de n elementos → $n + 1$ operaciones (considerando desde 0 hasta $n - 1$).
- 🕒 **Costo total = $2(n + 1)$**

3. Bucle `for i in range(...)`

- Se ejecuta **n veces** (una por cada índice de A).

Dentro del bucle:

🔸 `sum = sum + A[i]`

- **Acceso a A[i]** → 1 operación
- **Suma** → 1 operación
- **Asignación** → 1 operación
- 🕒 **Total por iteración = 3 operaciones**
- 🟥 **Total del bucle = $3n$**

Desglose alternativo (más detallado aún):

Si consideramos operaciones a nivel de bytecode (lo que realmente ejecuta Python):

1. `FOR_ITER` (obtener siguiente valor): 1 op
2. Comparación para término: 1 op
3. `LOAD_FAST` para `sum`: 1 op
4. `LOAD_FAST` para `A`: 1 op
5. `LOAD_FAST` para `i`: 1 op
6. `BINARY_SUBSCR` (acceso A[i]): 1 op
7. `BINARY_ADD`: 1 op
8. `STORE_FAST` para `sum`: 1 op
9. `JUMP_ABSOLUTE` (volver al inicio): 1 op

Teorema Maestro

El Teorema Maestro transforma una recurrencia compleja en una estimación directa de complejidad, simplificando el análisis de algoritmos recursivos.

Rocha (2025)

Definición 1.4.1: Función simple

Se dice que la función $T : \mathbb{N} \to \mathbb{R}_{\geq 0}$ es *simple* si satisface las siguientes condiciones para $n \in \mathbb{N}$:

- Caso base: para n suficientemente pequeño,

  $$T(n) \in O(1).$$

- Caso recurrente: para los demás valores de n y constantes $a, b, k \in \mathbb{R}_{\geq 0}$,

  $$T(n) \leq a \cdot T(n/b) + O(n^k)$$

  tales que:
  - $a \geq 1$ representa la cantidad de llamados recurrentes hechos por el algortitmo asociado a T y
  - $b > 1$ representa el factor por el cual se reduce el espacio en cada llamado recurrente de T.

Teorema Maestro

El Teorema Maestro transforma una recurrencia compleja en una estimación directa de complejidad, simplificando el análisis de algoritmos recursivos.

Rocha (2025)

Definición 1.4.1: Función simple

Se dice que la función $T : \mathbb{N} \to \mathbb{R}_{\geq 0}$ es *simple* si satisface las siguientes condiciones para $n \in \mathbb{N}$:

- Caso base: para n suficientemente pequeño,

  $$T(n) \in O(1).$$

- Caso recurrente: para los demás valores de n y constantes $a, b, k \in \mathbb{R}_{\geq 0}$,

  $$T(n) \leq a \cdot T(n/b) + O(n^k)$$

  tales que:
  - $a \geq 1$ representa la cantidad de llamados recurrentes hechos por el algortitmo asociado a T y
  - $b > 1$ representa el factor por el cual se reduce el espacio en cada llamado recurrente de T.

El caso base en una función simple indica que una vez el tamaño de la entrada es suficientemente pequeño como para no hacer llamados recurrentes, entonces el problema se puede resolver en tiempo/espacio constante (i.e., $O(1)$). El caso recurrente asume que se recurre sobre a instancias similares a la instancia dada, pero de tamaño reducido en una fración b, y que estos llamados requieren $O(n^k)$ tiempo/espacio para ser consolidados como respuesta de la instancia inicial.

Teorema Maestro

- ¿Es una función simple?
- ¿Cuántos llamados recursivos? -> a
- ¿Cómo se reduce el espacio de búsqueda? -> b
- ¿Cuál es la complejidad del caso base?
- ¿Cuál es la complejidad del proceso del caso recursivo? -> k

```
def f(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    izquierda = f(arr[:mid])
    derecha = f(arr[mid:])

    return g(izquierda, derecha)

def g(izq, der):
    resultado = []
    i = j = 0

    while i < len(izq) and j < len(der):
        if izq[i] <= der[j]:
            resultado.append(izq[i])
            i += 1
        else:
            resultado.append(der[j])
            j += 1

    resultado.extend(izq[i:])
    resultado.extend(der[j:])
    return resultado
```

Rocha (2025)

Teorema Maestro

Teorema 1.4.1: Teorema Maestro

Si $T : \mathbb{N} \to \mathbb{R}_{\geq 0}$ es una función simple con parámetros $a \geq 1$, $b > 1$ y $k \geq 0$, entonces para $n \in \mathbb{N}$:

$$T(n) \in \begin{cases} O(n^k \log n) & , \text{ si } a = b^k \\ O(n^k) & , \text{ si } a < b^k \\ O(n^{\log_b a}) & , \text{ si } a > b^k. \end{cases}$$

Teorema Maestro

🧠 Idea central del Teorema Maestro

- El **Teorema Maestro** permite resolver ecuaciones de recurrencia del tipo:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + \Theta(n^k)$$

- Aquí:
  - a: número de subproblemas generados.
  - b: factor por el cual se reduce el tamaño del problema.
  - n^k: trabajo adicional fuera de la recursión (por nivel).
  - Se compara a con b^k para decidir la complejidad final.

$$T(n) \in \begin{cases} O(n^k \log n) & , \text{si } a = b^k \\ O(n^k) & , \text{si } a < b^k \\ O(n^{\log_b a}) & , \text{si } a > b^k. \end{cases}$$

Teorema Maestro

📊 Casos del Teorema Maestro (intuición simple)

🔹 **Caso 1:** $a < b^k$

- **Menos subproblemas** de lo que costaría resolverlos por completo.
- El **trabajo fuera de la recursión (nivel superior)** domina.
- **Ejemplo de pensamiento:** el trabajo es más pesado al inicio que en los niveles más profundos.
- 🔚 **Resultado:**

$$T(n) = \Theta(n^k)$$

Teorema Maestro

Caso 2: $a = b^k$

- El **trabajo por nivel** es igual en todos los niveles.
- **Cada nivel hace trabajo** $\Theta(n^k)$ y hay $\log_b n$ niveles.
- **Resultado:**

$$T(n) = \Theta(n^k \cdot \log n)$$

Teorema Maestro

🔹 **Caso 3:** $a > b^k$

- Hay **más subproblemas** de los que se pueden manejar eficientemente.
- El **trabajo profundo** (últimos niveles) domina el costo total.
- Se produce una **explosión de subproblemas.**
- 🔚 **Resultado:**

$$T(n) = \Theta(n^{\log_b a})$$